File upload implementations

Outline:

- File upload implementations
  - Multipart form data
  - File type validation and size limits
  - Progress tracking for uploads
  - Drag and drop file upload interfaces

Multipart Form Data

Multipart form data is a data type used for uploading files or other binary data alongside text data in a single HTTP request.

How Multipart Form Data Works

When a form with `enctype="multipart/form-data"` is submitted, the browser:

1. Creates a unique boundary string to separate different form fields
2. Formats each form field with headers and content
3. Sends all data in a single HTTP request

Using React Hook Form

In React, libraries like **React Hook Form** facilitate handling such forms efficiently.

```
import { useForm } from 'react-hook-form';

function UploadForm() {
  // Initialize react-hook-form with default configuration
  const { register, handleSubmit } = useForm();

  // Handles form submission and file upload
  const onSubmit = async (data) => {
    // Create FormData object to handle file uploads
    const formData = new FormData();

    // Add file to FormData (using first file if multiple selected)
    formData.append('file', data.file[0]);

    // Add name field to FormData
    formData.append('name', data.name);

    // Send multipart/form-data request to server
    const response = await fetch('/api/upload', {
      method: 'POST',
      body: formData,
    });
    // Handle response
  };

  return (
    <form onSubmit={handleSubmit(onSubmit)}>
      {/* Name input field */}
      <input {...register('name')} />

      {/* File input field */}
      <input type="file" {...register('file')} />

      {/* Submit button */}
      <button type="submit">Upload</button>
    </form>
  );
}
```

The flow for the above is:

1. Form Submission:
    - When the user clicks the submit button, the form triggers the `handleSubmit` function
    - This wraps our onSubmit function with form validation and error handling

2. Data Processing:
    - React Hook Form collects data from registered inputs
    - The `FormData` API is used to create a special object that can handle file uploads
    - Files are appended to FormData using `append('file', data.file[0])`
    - Additional form data (like the name field) is appended separately

3. Server Communication:
    - The FormData object is sent as a multipart/form-data request
    - This format is specifically designed for file uploads
    - The server can parse both the file and additional form data separately

Next.js API Route for Handling Uploads

You'll need a server endpoint to receive the file:

```
// pages/upload.js
import { IncomingForm } from 'formidable';
import { promises as fs } from 'fs';

// Disable Next.js body parsing for this route
export const config = {
  api: {
    bodyParser: false,
  },
};

export default async function handler(req, res) {
  // Check if request method is POST
  if (req.method !== 'POST') {
    return res.status(405).json({ error: 'Method not allowed' });
  }

  // Create new formidable form parser
  const form = new IncomingForm();

  // Parse multipart form data
  form.parse(req, async (err, fields, files) => {
    // Handle parsing errors
    if (err) return res.status(500).json({ error: err.message });

    // Get the uploaded file
    const file = files.file[0];

    // Read the uploaded file data
    const data = await fs.readFile(file.filepath);

    // Save file to uploads directory
    await fs.writeFile(`./public/uploads/${file.originalFilename}`, data);

    // Return success response
    return res.status(200).json({ message: 'File uploaded successfully' });
  });
```

The flow for the above is:
Initial Request Handling:

- When a POST request arrives, Next.js first checks the method
- The `bodyParser: false` configuration is crucial because:
  - It prevents Next.js from parsing the `multipart/form-data`
  - Allows Formidable to handle the raw request body
  - Preserves the file data in its original form

2. Form Parsing Process:
   - Formidable creates a parser for the multipart form data
   - The parser extracts both files and fields from the request
   - This happens asynchronously through the `form.parse()` callback

3. File Processing:
   - Once parsed, the file is temporarily stored on disk
   - The filepath property points to this temporary location
   - The file is read using the filesystem promises API
   - It's then written to the final destination in the public/uploads directory

4. Error Handling:
   - Invalid methods return a 405 status
   - Parsing errors return a 500 status
   - Successful uploads return a 200 status
   - All errors include descriptive messages

File Type Validation and Size Limits

1. Type validation using MIME types:

```
const ALLOWED_TYPES = ["image/jpeg", "image/png", "application/pdf"];
const isValidType = ALLOWED_TYPES.includes(file.type);
```

2. Size validation with a maximum limit:

```
const MAX_FILE_SIZE = 5 * 1024 * 1024; // 5MB
const isValidSize = file.size <= MAX_FILE_SIZE;
```

Progress Tracking

For large files, showing upload progress improves user experience.
Progress tracking is implemented using the Progress component and simulated upload progress:

```
// Simulates a file upload progress with visual feedback
const simulateUpload = () => {
  // Set the uploading state to true to show the progress UI
  setIsUploading(true);

  // Create an interval that updates every 500ms to simulate progress
  const interval = setInterval(() => {
    // Update progress state using a functional update to ensure we have the
latest value
    setProgress((prev) => {
      // Check if we've reached 100% completion
      if (prev >= 100) {
        // Clean up the interval to prevent memory leaks
        clearInterval(interval);
        // Return 100 to ensure we don't exceed maximum
        return 100;
      }
      // Increment progress by 10% each interval
      return prev + 10;
    });
  }, 500);
};
```

Drag and Drop Interface

The drag-and-drop functionality uses native browser events:

```
// Handles file drop events from drag-and-drop operations
const handleDrop = async (e: React.DragEvent<HTMLDivElement>) => {
  // Prevent default browser behavior for drag events
  e.preventDefault();

  // Convert the FileList from the drag event to an array
  const droppedFiles = Array.from(e.dataTransfer.files);

  // Process the dropped files using the existing file processing logic
  processFiles(droppedFiles);
};
```

Refer here for practice: https://github.com/syangche/React_Practicals.git

Resources:

React Hook Form - Multipart Form Data File Uploads:
https://claritydev.net/blog/react-hook-form-multipart-form-data-file-uploads

MDN - Using Form Data:
https://developer.mozilla.org/en-US/docs/Web/API/FormData/Using_FormData_Objects

React Dropzone Documentation:
https://react-dropzone.js.org/

MDN - Drag and Drop API:
https://developer.mozilla.org/en-US/docs/Web/API/HTML_Drag_and_Drop_API

Axios Documentation:
https://axios-http.com/docs/intro